



A

Report on

Smart Traffic Signal Optimization using Game Theory

Prepared For

Data Science

Complex Engineering Problem 1 (CEP) · SDG 11: Sustainable Cities and Communities

Team G2 · Batch B1

SIES GRADUATE SCHOOL OF TECHNOLOGY NERUL,

NAVI MUMBAI 400706

ACADEMIC YEAR 2025-26



A

Report on

Smart Traffic Signal Optimization using Game Theory

Prepared For

Data Science · CEP

Submitted by: Team G2 (Batch B1)

Bharath Rathinasapabathy	123A8012
Gregory	123A8020
Harshwardhan Ahire	123A8004

Department of Artificial Intelligence & Data Science

Under the guidance of

[Name of guide]

SIES GRADUATE SCHOOL OF TECHNOLOGY NERUL,

NAVI MUMBAI 400706

ACADEMIC YEAR 2025-26

Acknowledgement

We, Team G2, express our sincere gratitude to our guide, **[Name of guide]**, for the direction, feedback and encouragement given throughout this project. Framing the work as a Complex Engineering Problem aligned with a Sustainable Development Goal pushed us to build a complete, testable system rather than a classroom demonstration.

We thank the Department of Artificial Intelligence & Data Science and SIES Graduate School of Technology, Nerul, for the laboratory facilities and the academic environment that made this work possible. We also thank our classmates, whose questions during the interim presentation helped us sharpen the problem statement and the evaluation.

Finally, we acknowledge the open-source communities behind JavaScript, Python, Node.js, Matplotlib and Playwright. Their tools allowed three students to build, test and document a complete multi-agent traffic simulator on ordinary laptops.

Team G2

Abstract

Traffic signals in most Indian cities run on fixed timers that cannot see the queues in front of them. Vehicles idle at red lights over empty cross streets, queues spill back into neighbouring junctions, and ambulances wait like everyone else. This report presents **Smart Traffic Signal Optimization using Game Theory**, a multi-agent traffic management system in which every signalised junction is a rational player that competes for green-light allocation.

Each junction chooses between North–South and East–West green. Its payoff rewards clearing queued and approaching vehicles and long-waiting traffic, and penalises switching, pushing vehicles into a neighbour whose queue is already full (spillback) and breaking a platoon that a neighbour is about to release. Decisions are a pure-strategy **Nash equilibrium** of this game, found every simulated second by best-response dynamics, under yellow, all-red, minimum-green and maximum-green safety rules. The system is evaluated in a microscopic traffic simulator with cars, two-wheelers, auto-rickshaws and buses on 2×2 and 3×3 grids, against a fixed-time controller and a selfish adaptive controller, using common random numbers, ten seeds and paired t-tests.

Version 2 turns the demonstration into a dynamic system. Users can block an approach, send an ambulance that triggers emergency preemption, change load and game weights while the simulation runs, enumerate every joint plan to measure how good the equilibrium is, view a signal timeline, run benchmarks with disruptions, sweep the spillback penalty, keep a run history, export CSV and JSON, and share a scenario as a link. The whole application is a single browser page with no server.

Across six traffic scenarios the Nash controller reduced average vehicle delay by **8% to 63%** against the fixed timer (every paired t-test $p < 0.001$). Ambulance trips fell from 50.1 s under fixed-time and 43.1 s without preemption to **31.8 s** with preemption. The equilibrium found by best-response dynamics reached 97% to 98% of the best possible joint payoff, and in every scenario the coupled Nash game was statistically indistinguishable from the selfish adaptive controller: most of the gain comes from reacting to queues at all. Demand is synthetic and the model is not calibrated or validated against real data, and the report states this openly. The project supports **SDG 11** (target 11.2, safe and sustainable transport), with links to SDG 3, SDG 9 and SDG 13.

Keywords: game theory, Nash equilibrium, best-response dynamics, traffic signal control, multi-agent systems, microscopic simulation, emergency vehicle preemption, paired t-test, SDG 11.

Table of Contents

Acknowledgement	1
Abstract	2
CHAPTER 1 Introduction.....	6
1.1 Background	6
1.2 Problem Statement	6
1.3 Motivation.....	6
1.4 Objectives	7
1.5 Alignment with the Sustainable Development Goals	7
1.6 Scope and Limitations.....	8
1.7 Organisation of the Report.....	8
CHAPTER 2 Literature Review and Technical Background	9
2.1 Traffic Signal Control	9
2.2 Game Theory Essentials	9
2.3 Multi-Agent Methods in Traffic	9
2.4 Emergency-Vehicle Preemption	10
2.5 Simulation and Statistical Evaluation	10
2.6 Research Gap and Contribution	10
CHAPTER 3 Requirements and Problem Analysis	11
3.1 Stakeholders	11
3.2 Functional Requirements	11
3.3 Non-Functional Requirements	12
3.4 Why This Is a Complex Engineering Problem (WP1 to WP7)	12
3.5 Project Plan and Work Breakdown.....	13
CHAPTER 4 System Design	14
4.1 Architecture Overview.....	14
4.2 Layers and Responsibilities	14
4.3 The Decision Cycle.....	15
4.4 State and Data	15
4.5 Safety Design	16
4.6 Reproducibility Design	16
CHAPTER 5 Game-Theoretic Methodology.....	17
5.1 The Signal Game.....	17
5.2 Observations	17
5.3 The Payoff Function	17
5.4 Equilibrium by Best-Response Dynamics	18
5.5 Baselines	18

5.6 Emergency-Vehicle Preemption	18
5.7 Blocked Approaches	19
5.8 Exact Equilibrium Analysis	19
5.9 Equilibrium Efficiency.....	19
5.10 Worked Example	19
CHAPTER 6 Simulation Model and Experimental Design.....	21
6.1 Road Network	21
6.2 Vehicles and Car Following.....	21
6.3 Demand	21
6.4 Metrics	21
6.5 Experimental Design.....	22
CHAPTER 7 Implementation	23
7.1 Tools and Technologies	23
7.2 Project Structure.....	23
7.3 User Interface.....	24
CHAPTER 8 Testing and Results.....	34
8.1 Test Strategy	34
8.2 Controllers on Six Traffic Scenarios.....	34
8.3 Detailed Results for the Balanced 2×2 Scenario	36
8.4 Disruptions.....	36
8.5 Emergency Vehicle Preemption	37
8.6 Sensitivity to the Spillback Penalty λ	38
8.7 Quality of the Equilibrium	39
8.8 Threats to Validity and Limitations	40
CHAPTER 9 Social Impact, Ethics and Sustainability	41
9.1 Expected Impact.....	41
9.2 Ethical Considerations	41
9.3 Sustainability.....	41
CHAPTER 10 Mapping to Course Outcomes, Programme Outcomes, Knowledge Profiles and WP.....	42
10.1 Course Outcome Mapping	42
10.2 Programme Outcome Mapping.....	42
10.3 Knowledge Profile (WK) Mapping	43
10.4 Complex Problem-Solving Attributes (WP) Mapping	44
CHAPTER 11 Conclusion and Future Scope	45
11.1 Conclusion	45
11.2 Future Scope	45
References.....	46

Appendix A: Running the Project.....	47
Appendix B: Parameters	48

CHAPTER 1

Introduction

1.1 Background

Traffic congestion is one of the most visible failures of urban infrastructure. In dense cities such as Mumbai, vehicles lose a large share of their trip time at signalised junctions. A fixed-time signal gives each road a preset green regardless of demand. It is cheap and predictable, but it wastes green on empty approaches, cannot respond to a stalled vehicle or a cricket-match crowd, and cannot give way to an ambulance.

Adaptive control reads detectors and adjusts the signal. The hard part is coordination: a junction that serves its own queue greedily may push vehicles into a neighbour whose road is already full, which blocks that neighbour and eventually blocks the junction itself. Each signal controls only its own phase, yet its outcome depends on the neighbours' phases. That is the structure of a non-cooperative game, which is why game theory is a natural language for the problem.

This project, **Smart Traffic Signal Optimization using Game Theory**, models every signalised junction as a rational player, computes each decision as a Nash equilibrium, and tests the idea in a microscopic traffic simulator. Version 2 adds the dynamic features that make the system usable as a live decision-support tool: disruptions, emergency preemption, exact equilibrium analysis, and reproducible experiments.

1.2 Problem Statement

Complex Engineering Problem, option 1: Smart Traffic Signal Optimization using Game Theory. Design and implement a multi-agent traffic management system where each traffic signal acts as a rational player competing for green-light allocation, to minimise overall congestion and vehicle waiting time.

1.3 Motivation

- **Waiting is waste.** Every second a vehicle idles costs time and fuel, and adds emissions in already polluted streets.
- **Local decisions, network consequences.** A greedy signal can hurt its neighbours. A model that captures this interaction is needed, not just a queue-length rule.
- **Emergencies.** An ambulance should not wait through a full signal cycle behind a queue.
- **Disruptions are normal.** Breakdowns, road works and processions block approaches. A controller should notice and adapt.

- **Evidence over claims.** Any improvement should be measured against a baseline, on identical traffic, with a statistical test.

1.4 Objectives

1. Model each signalised junction as a rational agent with a strategy set and a payoff function.
2. Compute each signal decision as a pure-strategy Nash equilibrium of the resulting non-cooperative game.
3. Build a microscopic traffic simulator with mixed Indian traffic, to test the controller under safe, realistic rules.
4. Compare the game-theoretic controller with a fixed-time controller and a selfish (non-cooperative) adaptive controller.
5. Evaluate with data-science methods: repeated runs, common random numbers, confidence from paired t-tests, and several load scenarios.
6. Make the system dynamic: live disruptions, emergency-vehicle preemption, tunable game weights, and exact equilibrium analysis.
7. Provide reproducible evidence: seeded runs, exportable results, saved history, and automated tests.
8. Document limits honestly, including what the data does and does not show.

1.5 Alignment with the Sustainable Development Goals

Sustainable Development Goal 11 aims to "make cities and human settlements inclusive, safe, resilient and sustainable" [1]. The project contributes as follows:

Table 1: Sustainable Development Goals addressed by the project

Goal / target	Statement (summary)	Contribution of this project
SDG 11.2	Provide access to safe, affordable, accessible and sustainable transport systems and improve road safety.	Lower delay and fewer stops on shared roads; emergency vehicles cross the network sooner; safe signal timings are never relaxed.
SDG 11.6	Reduce the adverse per capita environmental impact of cities, including air quality.	Less idling time. The tool reports idling vehicle-hours and an illustrative CO ₂ estimate.
SDG 3.6	Halve road traffic deaths and injuries (supporting).	Faster ambulance response; safety-first design with yellow and all-red intervals.
SDG 9.1	Develop quality, reliable, resilient infrastructure (supporting).	A controller that adapts to blocked roads instead of failing silently.

Goal / target	Statement (summary)	Contribution of this project
SDG 13	Take urgent action on climate change (supporting).	Reducing idling lowers fuel burnt per trip.

1.6 Scope and Limitations

In scope: a 2×2 or 3×3 grid of signalised junctions; four vehicle types; three controllers; live disruptions (blocked approach, ambulance); exact equilibrium enumeration; benchmark, sweep, history and export; a single-page web application; automated tests.

Out of scope: pedestrians, left-turn phases, more than two phases per junction, multi-lane roads, real detector data, calibration against a real city, and validation against an established simulator such as SUMO. Demand is **synthetic**, so every delay figure describes the controllers inside our simulator and should not be read as a prediction for a real road network.

1.7 Organisation of the Report

Chapter 2 reviews related work and the techniques used. Chapter 3 analyses requirements, shows why this is a Complex Engineering Problem (attributes WP1 to WP7) and gives the project plan. Chapter 4 presents the system design. Chapter 5 describes the game-theoretic method in detail. Chapter 6 describes the simulation model and the experimental design. Chapter 7 describes the implementation and the user interface. Chapter 8 reports testing and results. Chapter 9 discusses social impact, ethics and sustainability. Chapter 10 maps the work to course outcomes, programme outcomes, knowledge profiles and complex-problem attributes. Chapter 11 concludes with future scope.

CHAPTER 2

Literature Review and Technical Background

2.1 Traffic Signal Control

Classical signal design sets cycle length and green splits from average flows. Webster [2] derived the cycle length that minimises delay at an isolated junction under uniform arrivals; fixed-time plans built this way are still common. **Actuated control** extends or truncates greens when detectors see vehicles. **Network-level adaptive systems** such as SCOOT and SCATS adjust splits and offsets from measured flows. A more recent line, **max-pressure control** [3], chooses the phase that relieves the largest difference between upstream and downstream queues and has provable stability properties under mild assumptions. Our selfish and Nash controllers belong to the same family of queue-driven, decentralised rules and are compared with a fixed-time baseline.

2.2 Game Theory Essentials

A **normal-form game** has a set of players, a set of actions for each player and a payoff for each player for every combination of actions [4]. A **pure-strategy Nash equilibrium** is an action profile in which no player can raise its own payoff by changing only its own action. A **best response** is the action that maximises a player's payoff given the others' actions. In **best-response dynamics**, players take turns replacing their action by a best response. If the sequence stops, the profile is a Nash equilibrium. In general, pure equilibria need not exist and the dynamics can cycle. A **potential game** [5] is a game in which a single function increases whenever any player improves by a unilateral change; best-response dynamics always converge in a finite potential game. Congestion games are the classical example. Our payoff is built from congestion-style terms, so we expect good convergence, but we **measure** it instead of assuming it: the simulator reports how often the dynamics reached equilibrium (100% in every experiment) and caps each decision at eight rounds.

The **price of anarchy** is the ratio between the best total outcome and the worst equilibrium; the **price of stability** compares the best total outcome with the best equilibrium. Because our payoffs can be negative, a ratio is not meaningful, so we report a normalised measure called equilibrium efficiency (Chapter 5.9).

2.3 Multi-Agent Methods in Traffic

Bazzan [6] surveys multi-agent systems and multi-agent reinforcement learning for traffic control, and notes that independent learners at each junction can fail to coordinate. Game-theoretic formulations make the interaction explicit and give a decision rule that can be

explained at the control room. Reinforcement learning can learn payoffs from data but needs training time, a simulator and care about safety; we list it as future work.

2.4 Emergency-Vehicle Preemption

Many signal systems offer preemption: when an emergency vehicle is detected, the signal ends the current phase early, observing yellow and all-red clearance, and holds green on the vehicle's approach. We reproduce the key behaviour (a priority term in the payoff and a shortened minimum green) without relaxing the clearance intervals.

2.5 Simulation and Statistical Evaluation

Microscopic simulators represent each vehicle and its interaction with the vehicle ahead. We use a simple safe-distance rule: each vehicle accelerates towards its desired speed but never faster than the speed from which it can still stop in the free distance ahead with a fixed braking rate [7]. For comparing controllers we use **common random numbers** [8]: for a given seed every controller sees identical arrival times, vehicle types and turning choices, so differences come from the controller alone and variance is reduced. Results are averaged over seeds and compared with a **paired two-sided t-test** [9].

2.6 Research Gap and Contribution

Table 2: Gaps addressed by this work

Gap in common practice	How this project addresses it
Fixed timers ignore queues	Detector-driven controllers cut average delay by 8% to 63% in our scenarios.
Local greed ignores neighbours	The payoff includes spillback and coordination terms; the effect is measured with a sweep of the penalty λ .
Claims of "optimal" without evidence	Exact enumeration of all joint plans measures how close the equilibrium is to the best joint outcome.
Emergency vehicles wait in queues	Preemption through the payoff and a shortened minimum green, with all clearance intervals kept.
Disruptions handled by hand	Blocked approaches are detected; controllers stop giving them green.
Results that cannot be reproduced	Seeded runs, common random numbers, CSV and JSON export, a one-command experiment script.

CHAPTER 3

Requirements and Problem Analysis

3.1 Stakeholders

Table 3: Stakeholders and their needs

Stakeholder	Needs	What the system provides
Commuters, bus and two-wheeler riders	Shorter, smoother trips with fewer stops.	Lower delay, fewer stops and higher throughput than a fixed timer.
Emergency services	Fast, safe passage.	Preemption that cuts ambulance trip time while keeping clearance intervals.
Traffic police / control room	To see what the signals are doing and why; to react to incidents.	Live payoff matrix, equilibrium analysis, event log, blocked-approach handling.
City planners	Evidence to compare control strategies.	Reproducible benchmark with statistics, CSV and JSON export, parameter sweep.
Examiners and students	A system they can explore and test.	One-file web app, shareable scenario links, automated tests, written report.

3.2 Functional Requirements

Table 4: Functional requirements

ID	Requirement	Module
FR1	Simulate a 2×2 or 3×3 grid of signalised junctions with mixed vehicles and left-hand traffic.	Engine
FR2	Generate demand with configurable rate, direction mix and a rush-hour wave, from a seed.	Engine
FR3	Run fixed-time, selfish and Nash controllers side by side on identical traffic.	Engine, UI
FR4	Compute each Nash decision by best-response dynamics under safety rules, and report whether equilibrium was reached.	Engine
FR5	Show the live payoff matrix and best-response rounds for any pair of neighbouring junctions.	UI
FR6	Let the user tune ω , σ , λ and κ while the simulation runs.	UI
FR7	Inject a blocked approach and an ambulance live, identically for all controllers.	Engine, UI
FR8	Enumerate all joint plans, count Nash equilibria, and report equilibrium efficiency.	Engine, UI
FR9	Run multi-seed benchmarks, with and without disruptions, with paired t-tests and a 95th-percentile delay.	UI
FR10	Sweep the spillback penalty and suggest the best value.	UI

ID	Requirement	Module
FR11	Export results as CSV and JSON, keep a run history, share scenarios as links.	UI
FR12	Provide light and dark themes and keyboard shortcuts for presenting.	UI

3.3 Non-Functional Requirements

- **Safety:** conflicting greens must be impossible; yellow, all-red and minimum green apply to every controller. Tests check zero red-light entries and zero vehicle overlaps.
- **Reproducibility:** the same seed gives the same traffic; adding analysis or visual mode does not change results (tested).
- **Performance:** three simulations and the exact equilibrium analysis run in real time at up to $8\times$ speed on a laptop.
- **Explainability:** every decision can be traced to a payoff matrix and a list of best-response rounds.
- **Portability:** a single HTML file that works offline; no installation, no server.
- **Honesty:** limitations are stated in the application and in this report.

3.4 Why This Is a Complex Engineering Problem (WP1 to WP7)

Complex Engineering Problems are characterised by the complex problem-solving attributes WP1 to WP7 of the Washington Accord graduate attributes. The table below relates each attribute to this project; a problem should satisfy WP1 and some of the others.

Table 5: Complex engineering problem attributes WP1 to WP7

WP	Attribute	Meaning	Evidence in this project
WP1	Depth of knowledge required	Cannot be resolved without in-depth knowledge at the level of WK3, WK4, WK5, WK6 or WK8.	Game theory (Nash equilibrium, best response, potential games), traffic-flow modelling, discrete-event simulation, statistics (paired t-test, common random numbers) and web engineering are all needed together.
WP2	Range of conflicting requirements	Wide-ranging or conflicting technical, engineering and other issues.	A junction's own benefit versus network benefit (λ); delay versus stability (switching cost σ); adaptivity versus safety timings; ambulance priority versus the wait of everyone else; realism versus a model small enough to explain.
WP3	Depth of analysis required	No obvious solution; abstract thinking and originality in analysis to formulate suitable models.	The payoff function, the coupling between neighbours, the equilibrium computation, the emergency term and the evaluation design were formulated and tested by experiments (Chapters 5, 6, 8).

WP	Attribute	Meaning	Evidence in this project
WP4	Familiarity of issues	Infrequently encountered issues.	Equilibrium selection in a coupled signal game, spillback in short mixed-traffic links and preemption inside a game-theoretic controller are beyond standard textbook exercises.
WP5	Extent of applicable codes	Outside problems encompassed by standards and codes of practice.	Yellow and all-red clearance and minimum and maximum green follow signal-timing practice (for example Webster [2] and IRC guidance); the simulation model itself is not covered by a code, and departures are stated in Chapter 6.
WP6	Extent of stakeholder involvement	Diverse groups with widely varying needs.	Commuters, bus riders, two-wheeler riders, emergency services, traffic police and city planners want different things from the same junction.
WP7	Interdependence	High-level problems with many components or sub-problems.	Demand, vehicle dynamics, detectors, four interacting signals, safety rules, disruptions and metrics all feed one another; a change to one payoff weight changes every other component's behaviour.

3.5 Project Plan and Work Breakdown

The project was organised as eight activities. The table lists each with its output. The simulator, the controllers and the tests were built first, so that every later claim could be measured.

Table 6: Work breakdown

Task	Activity	Output	Status
T1	Problem study and literature review	Problem statement, game model sketch, reference list	Done
T2	Traffic simulator	Vehicles, car-following, junction box rules, demand generator (src/engine.js)	Done
T3	Game model and controllers	Payoff, best-response dynamics, selfish and fixed baselines	Done
T4	Safety and correctness tests	Overlap and red-light invariants, regression run, feature checks (tests/)	Done
T5	Evaluation harness	Common random numbers, paired t-test, benchmark tab	Done
T6	Dynamic features (version 2)	Blocked approach, ambulance preemption, equilibrium enumeration, timeline, sweep, history, export, share link	Done
T7	User interface	Map, panels, payoff matrix, charts, themes, shortcuts (src/ui.js, style.css)	Done
T8	Report, presentation and repository	This report, presentation script, GitHub repository	Done

CHAPTER 4

System Design

4.1 Architecture Overview

The application is a closed loop. A demand generator feeds vehicles into a microscopic simulator. Detectors summarise what each approach looks like (queue, approaching flow, waiting time, emergency vehicle, blockage). A controller chooses phases; a phase machine applies them under safety rules, and the lights change what the vehicles do. Disruptions enter the simulator directly. A measurement layer records metrics, and a presentation layer draws everything in the browser.

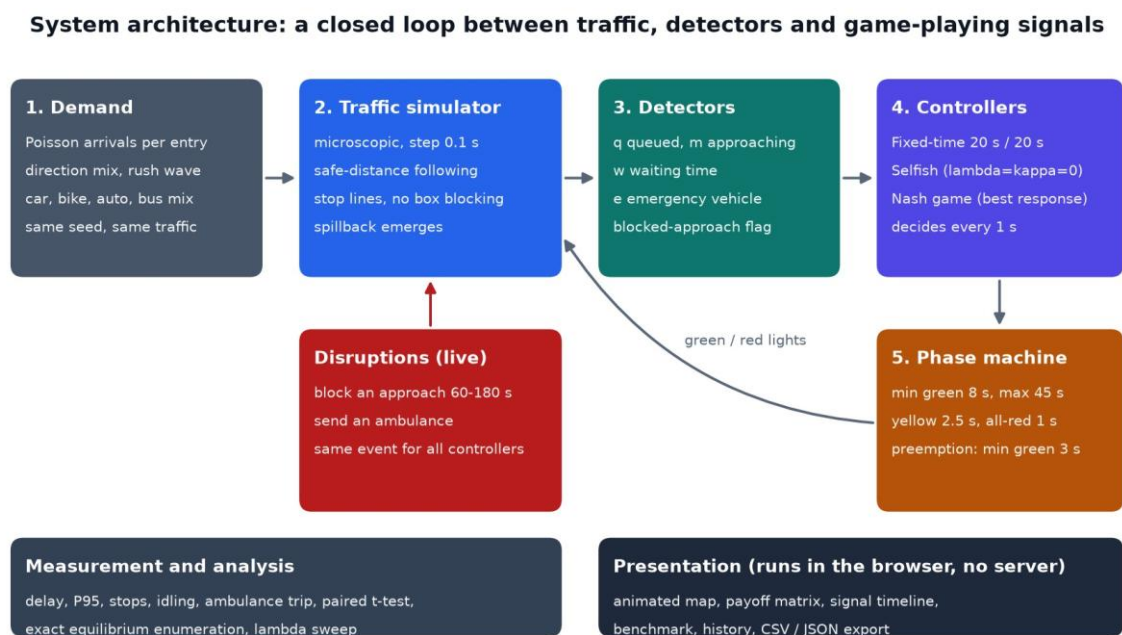


Figure 1: System architecture

4.2 Layers and Responsibilities

Table 7: Layers of the system

Layer	Technology	Responsibility
Presentation	HTML, CSS, JavaScript, Canvas	Map, panels, charts, themes, keyboard shortcuts, links.
Evaluation	JavaScript (statistics code)	Benchmark, paired t-test, sweep, history, CSV and JSON export.
Control	JavaScript	Fixed-time, selfish and Nash controllers; best-response dynamics; preemption; equilibrium enumeration.
Simulation	JavaScript	Vehicles, car-following, junction rules, demand, incidents, ambulance.

Layer	Technology	Responsibility
Tests and tooling	Node.js, Python, Playwright, Matplotlib	Invariant checks, feature checks, experiments, screenshots, figures, report build.

4.3 The Decision Cycle

Once per simulated second every adaptive controller runs one decision cycle (Figure 2). It reads the detector data, starts from the junctions' current phases (junctions already in yellow or all-red are committed to their next phase and cannot change), lets each free junction in turn pick its best response, repeats until a full sweep changes nothing, and finally applies the result through the phase machine.

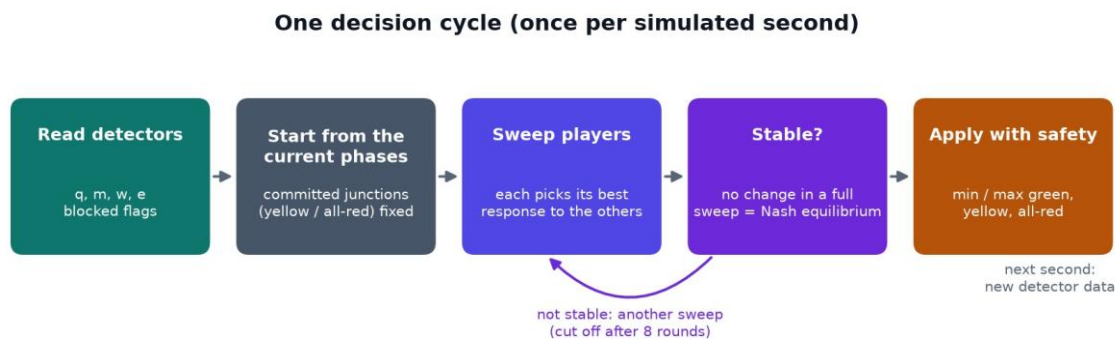


Figure 2: One decision cycle of the Nash controller

4.4 State and Data

Table 8: Main state of the simulator

Object	Contents
Junction	Row, column, phase (0 = North–South, 1 = East–West), status (green, yellow, all-red), timers.
Vehicle	Axis, lane, position, speed, length, type, colour, entry time, distance, stop counters, turning random numbers, emergency flag.
Detector reading (per junction, per approach)	q: queued vehicles near the stop line; m: vehicles approaching; w: accumulated waiting time of the queue; e: emergency vehicles approaching; blocked flag.
Incident	Junction, approach, start and end time.
Metrics	Exited trips, delay sums, stops, stopped time, queue integral, throughput, switches, ambulance trips, equilibrium statistics.
Time series	Samples every 5 s for the live chart; one phase sample per second for the timeline.

4.5 Safety Design

- **One green axis per junction.** Only one of the two phases can be green; a yellow (2.5 s) and an all-red (1 s) interval separate phases for every controller.
- **Minimum and maximum green.** 8 s and 45 s apply to all adaptive control. Preemption may shorten the minimum green to 3 s but never skips yellow or all-red.
- **No box blocking.** A vehicle does not enter a junction unless there is room to leave it.
- **Safe following.** Vehicles never move faster than the speed from which they can stop in the free distance ahead.
- **Automated checks.** Two invariant scripts count vehicle overlaps and stop-line crossings against red, on three controllers over 6,000 steps (10 simulated minutes); both counts are zero.

4.6 Reproducibility Design

Random numbers come from a seeded generator, and every random decision about a vehicle (type, colour, speed factor, turning choices) is drawn when the vehicle is created. Controllers therefore never consume random numbers, so for a given seed the three controllers see identical arrival times, vehicle types and turning choices. Emergency vehicles use fixed values instead of random draws. A test confirms that switching on equilibrium analysis and visual mode does not change the simulation outcome.

CHAPTER 5

Game-Theoretic Methodology

5.1 The Signal Game

The game is $\Gamma = (N, \{A_i\}, \{u_i\})$. The **players** N are the signalised junctions: four on a 2×2 grid and nine on a 3×3 grid. The **actions** are $A_i = \{ \text{North–South green, East–West green} \}$. Each player chooses which road gets the next green. The **payoffs** u_i depend on the player's own action and on its neighbours' actions, which makes the game genuinely coupled.

5.2 Observations

For each approach $k \in \{N, S, E, W\}$ of junction i the detectors give q_k (stopped vehicles within 150 px, about 45 m, of the stop line), m_k (moving vehicles within 260 px, about 78 m), w_k (the sum of the time each queued vehicle has already been stationary) and e_k (emergency vehicles within 260 px). A blocked approach is flagged.

5.3 The Payoff Function

The payoff of junction i for choosing phase a , given the others' phases, is

$$u_i(a, a_{-i}) = \sum_{k \in a} (q_k + 0.4 m_k + \omega w_k / 30 + E e_k) - \sigma \cdot I[a \neq \text{current}] \\ - \lambda \sum_{j \in \text{down}(i, a)} q_{j \leftarrow i} (1 - 0.7 \cdot I[a_j = a]) \\ + \kappa \sum_{j \in \text{up}(i, a)} 0.6 q_{j \rightarrow i} \cdot I[a_j = a]$$

Table 9: Terms of the payoff function

Term	Default	Meaning
$q_k + 0.4 m_k$	weights 1 and 0.4	Gain for vehicles that can be cleared now (queued) and soon (approaching).
$\omega w_k / 30$	$\omega = 1.0$	Fairness: long-waiting queues pull the green towards them.
$E e_k$	$E = 60$	Emergency vehicle waiting on the phase (zero unless preemption is on). Blocked approaches contribute nothing.
σ	$\sigma = 2.0$	Switching cost (lost time): discourages flip-flopping.
λ	$\lambda = 0.6$	Spillback penalty: reluctance to send vehicles into a neighbour whose queue on that link is full, reduced by 70% if the neighbour is also serving that road.
κ	$\kappa = 0.5$	Coordination bonus: reward for matching a neighbour that is releasing a platoon towards the junction on the same road.

5.4 Equilibrium by Best-Response Dynamics

A profile a is a *pure-strategy Nash equilibrium* if $u_i(a_i, a_{-i}) \geq u_i(a_{-i}, a_{-i})$ for every player i and every action a_{-i} . The controller finds one as follows:

1. Read the detector data q, m, w, e and the blocked flags.
2. Start from the current phases. Junctions in yellow or all-red are committed to their next phase and take no part.
3. Sweep the free junctions in a fixed order. Each picks the action with the higher payoff given everyone else's current action; a tie keeps the current action.
4. Repeat sweeps until a whole sweep changes nothing. That profile is a pure Nash equilibrium. A cycle is cut off after 8 rounds.
5. Apply the profile through the phase machine: a junction may start switching only if its green has lasted at least the minimum green (8 s, or 3 s when preempting for an ambulance), and a junction is forced to switch after the maximum green (45 s) if the other road has demand.

Because the payoff is built from congestion-style terms, the dynamics settle quickly: in our experiments they reached equilibrium in 100% of decisions, in about 1.6 to 1.7 rounds on average in the 2×2 balanced scenario. This is an empirical observation for this payoff design, not a proof. The simulator reports the convergence rate for every run.

5.5 Baselines

Table 10: Controllers compared

Controller	Rule
Fixed-time	20 s North–South, 20 s East–West, with yellow and all-red between. No sensing; cannot see blockages or ambulances.
Selfish adaptive	The same game with $\lambda = \kappa = 0$. Each junction maximises only its own local benefit; the best responses are independent.
Nash game	The full coupled payoff above; the equilibrium of the whole network.

5.6 Emergency-Vehicle Preemption

When an ambulance is within 260 px of a stop line, the term $E e_k$ ($E = 60$) is added to the payoff of the phase that serves its approach. The value is far above typical queue sums, so the best response flips to the ambulance's road. In the phase machine the minimum green is relaxed from 8 s to 3 s, but only for a junction whose current phase does **not** serve the ambulance, and yellow (2.5 s) and all-red (1 s) are always observed. The selfish and Nash controllers both preempt (the user can switch preemption off); fixed-time has no sensing and cannot.

5.7 Blocked Approaches

An incident closes one approach of one junction for 60 to 180 s: vehicles queue at that stop line and cannot cross. The detector flags the approach as blocked. The payoff gain is zero for a blocked approach (a green there would clear nothing), and the maximum-green demand test ignores it. The queue on the blocked link is still visible to the upstream neighbour through the spillback term, so the neighbour becomes reluctant to send more traffic into it. Fixed-time cannot see the flag and keeps granting green to the blocked stop line.

5.8 Exact Equilibrium Analysis

To measure the quality of the equilibrium, every second the simulator enumerates all 2^F joint plans of the F free junctions (16 for a 2×2 grid; at most 512 for 3×3). For each plan it computes the total payoff $W = \sum_i u_i$, and checks the Nash condition for every free player. It then records the number of pure equilibria, the best and worst total payoff, and the rank of the plan chosen by best-response dynamics.

5.9 Equilibrium Efficiency

Payoffs can be negative, so a ratio-based price of anarchy is not meaningful. We define

$$\text{efficiency} = (W(\text{chosen}) - W(\text{worst})) / (W(\text{best}) - W(\text{worst}))$$

which is 100% when the equilibrium found is also the best joint plan and 0% if it is the worst. The measure is reported live on the Game model tab and averaged over decisions in the experiments.

5.10 Worked Example

Figure 3 shows the live payoff matrix for junctions J1 and J2, which share the East–West road, at one instant in a 2×2 run. J1’s payoff is -0.3 if it serves North–South and 0.8 if it serves East–West, whatever J2 does, so East–West is J1’s best response in both columns. J2 earns 2.0 serving North–South and -1.6 serving East–West, whatever J1 does, so North–South is its best response in both rows. The only cell where both payoffs are underlined is (J1: East–West, J2: North–South): the unique pure Nash equilibrium, with payoffs 0.8 and 2.0 . When both junctions’ best responses do not depend on the other’s action the game at that instant is effectively decoupled; the coupling terms λ and κ matter when neighbouring links are close to full.

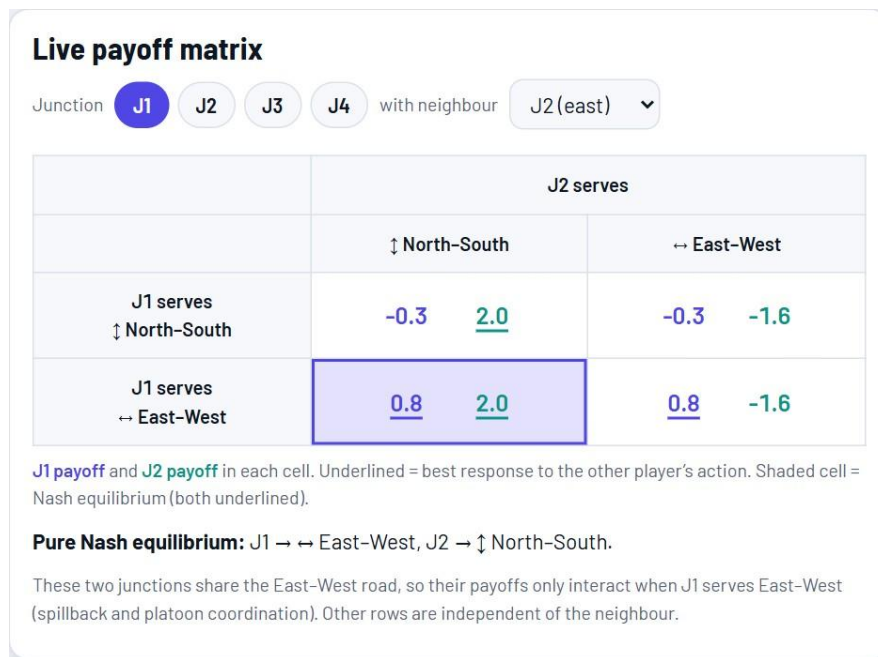


Figure 3: Live payoff matrix for J1 and J2 with the pure Nash equilibrium shaded



Figure 4: Best-response rounds at the latest decision: the profile is stable after one change round

CHAPTER 6

Simulation Model and Experimental Design

6.1 Road Network

An $N \times N$ grid of junctions ($N = 2$ or 3) connected by two-way roads with one lane per direction and left-hand traffic. Vehicles enter at the $4N$ border roads and leave at the far border. About a quarter turn left or right at each junction they cross (turning probability 0.25).

6.2 Vehicles and Car Following

Table 11: Vehicle types

Type	Share	Length	Notes
Car	55%	18 px	Free speed factor about 1.0
Two-wheeler	25%	10 px	Slightly faster (1.05)
Auto-rickshaw	12%	14 px	Slower (0.9)
Bus	8%	32 px	Slowest (0.85), longest
Ambulance	on demand	20 px	Fast (1.25); never turns

Each vehicle accelerates (16 px/s^2) towards its desired speed (about 45 px/s) but never faster than $\sqrt{(2 b d)}$, where $b = 26 \text{ px/s}^2$ is the braking rate and d the free distance to the vehicle ahead or to the stop line it must obey. A 5 px minimum gap is kept. Scale: $1 \text{ px} \approx 0.3 \text{ m}$, so the free speed is about 49 km/h ; the time step is 0.1 s .

6.3 Demand

Arrivals at each border road follow a Poisson process with a configurable rate (2 to 14 vehicles per minute), a direction mix (East–West versus North–South, $\pm 60\%$) and an optional rush-hour wave in which the rate varies between 0.5 and 1.5 times the base over 15 minutes. A vehicle that cannot enter because the road end is occupied waits in an entry queue and is counted in the delay.

6.4 Metrics

Table 12: Metrics

Metric	Definition
Average delay (s)	Trip time minus free-flow time, over finished trips, vehicles still on the map and vehicles waiting to enter. A controller cannot look good by leaving cars outside.
95th percentile delay (s)	The delay below which 95% of vehicles fall: a fairness / worst-case view.

Metric	Definition
Vehicles stopped	Time average of the number of vehicles moving slower than 2 px/s.
Stops per trip	Number of stop-and-go events per finished trip.
Throughput (trips/min)	Finished trips per minute.
Idling time (vehicle-hours)	Sum of the stopped time of all vehicles. An illustrative CO ₂ estimate multiplies it by 0.8 L/h idle fuel burn and 2.31 kg CO ₂ per litre of petrol; it is an indicator, not a measurement.
Ambulance trip (s)	From dispatch to leaving the network, including any wait to enter.
Equilibrium efficiency	See Chapter 5.9.

6.5 Experimental Design

- **Seeds and duration.** Ten seeds (1000, 1101, ..., 1909), each 15 simulated minutes (9,000 steps).
- **Common random numbers.** For a given seed all controllers see identical arrivals and vehicle properties.
- **Scenarios.** 2×2 balanced (8 vehicles/min per entry), 2×2 East–West heavier by 45%, 2×2 heavy (11/min), 2×2 rush-hour wave, 3×3 balanced and 3×3 heavy.
- **Disruptions.** Blocked approaches at 240 s (J1, from the north) and 540 s (last junction, from the south), 120 s each; ambulances at 180 s, 480 s and 720 s on different roads. The same events are injected for every controller.
- **Statistics.** Paired two-sided t-test across seeds; $p < 0.05$ is treated as significant. Standard deviations over seeds are reported.
- **Game weights.** $\omega = 1.0$, $\sigma = 2.0$, $\lambda = 0.6$, $\kappa = 0.5$ unless a sweep varies them.

Everything in Chapter 8 is produced by one command, **node tests/experiments.js**, which writes docs/results.json; the report tables are generated from that file.

CHAPTER 7

Implementation

7.1 Tools and Technologies

Table 13: Tools used

Area	Tool	Use in the project
Language	JavaScript (ES2020)	Simulator, controllers, statistics and user interface in one code base.
Rendering	HTML5 Canvas, CSS	Animated map, charts, themes, responsive layout.
Build	Python 3 (build.py)	Bundles src/ into one self-contained index.html.
Testing	Node.js	Invariant checks, feature checks, experiments.
Screenshots and figures	Playwright, Matplotlib, Pillow	Reproducible screenshots and result charts for this report.
Report	docx (Node), Times New Roman layout	Generated from the results file so numbers cannot drift.
Version control	Git, GitHub	https://github.com/bharath050305/traffic-signal-game-theory

7.2 Project Structure

Table 14: Project structure

Path	Contents
index.html	Built single-file application (generated).
src/engine.js	Simulator, controllers, best-response dynamics, equilibrium enumeration, disruptions, statistics.
src/ui.js	Rendering, panels, benchmark, sweep, history, export, shortcuts, links.
src/body.html, src/style.css	Page markup (including the in-app report) and light and dark styling.
build.py	Bundles the sources into index.html.
tests/test.js, inv.js, inv2.js	Headless benchmark; overlap and red-light invariants.
tests/features.js	Checks for incidents, preemption and equilibrium analysis (17 checks).
tests/experiments.js	Reproduces every number in the report; writes docs/results.json.
docs/	This report, its build scripts, figures and screenshots, and the presentation script.

7.3 User Interface

The interface has four tabs: Live simulation, Game model, Benchmark and Project report. The figures below are captured from the running application by a script, in a reproducible state.

Live simulation

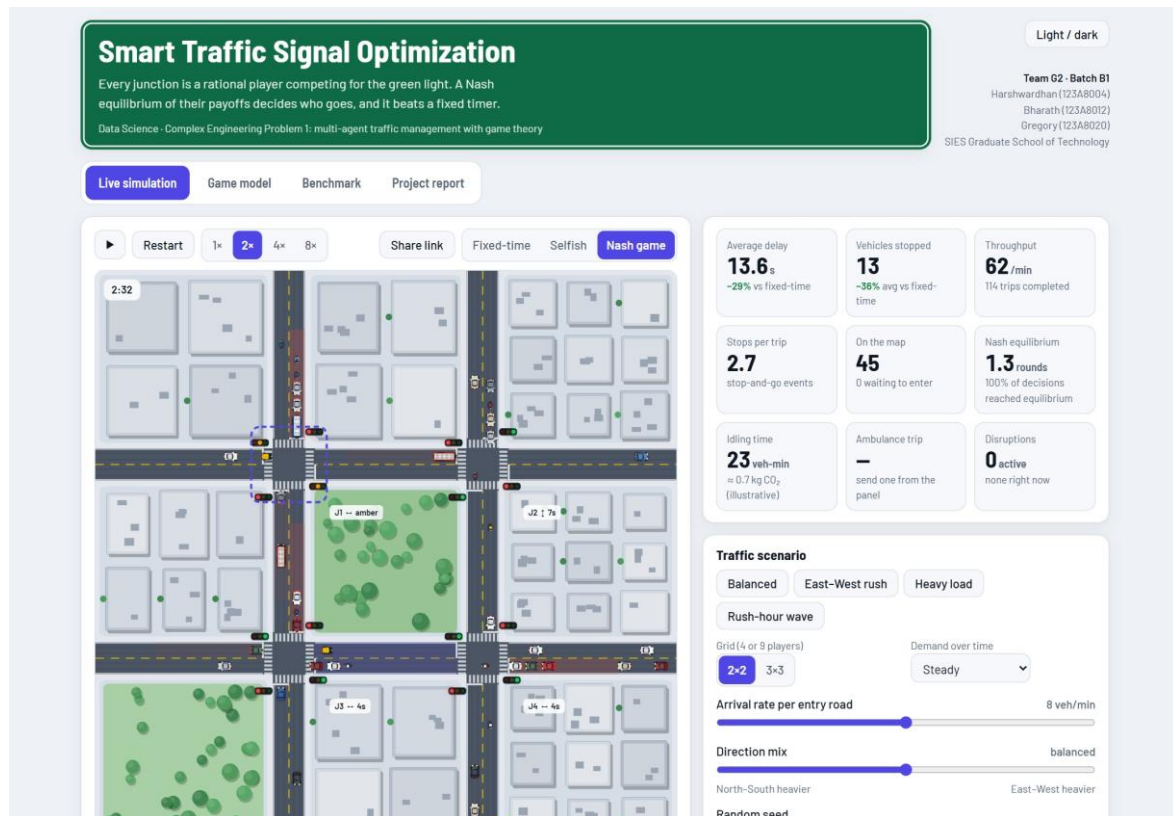


Figure 5: Live simulation tab: map, key indicators, scenario controls and the disruption panel

The map shows signal heads, queue heat, green corridors (two junctions serving the same road) and the controller chosen by the user. The three controllers run side by side on identical arrivals in the background, and the chart compares them live.

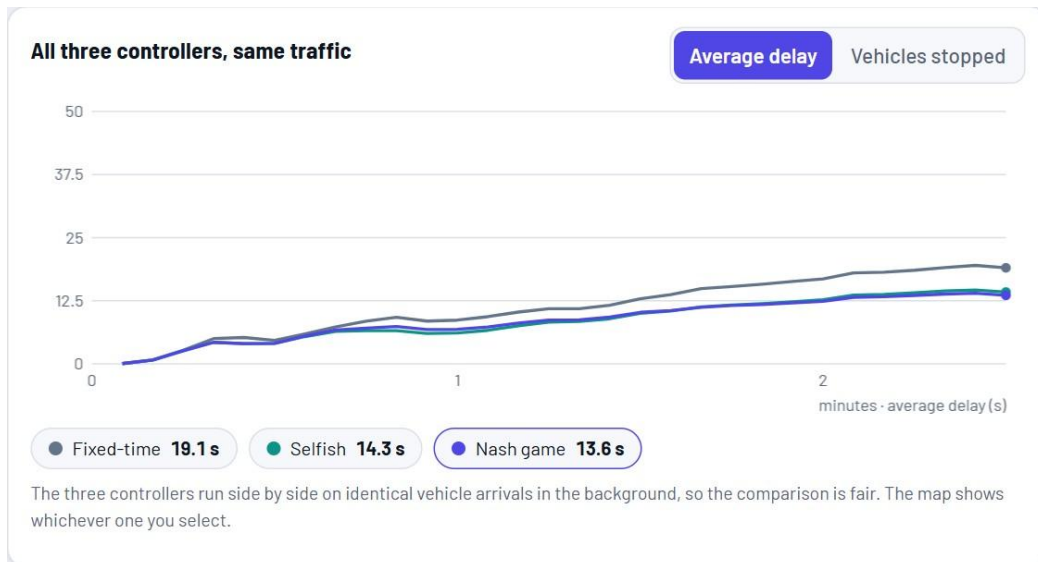


Figure 6: Average delay of the three controllers over time on identical traffic

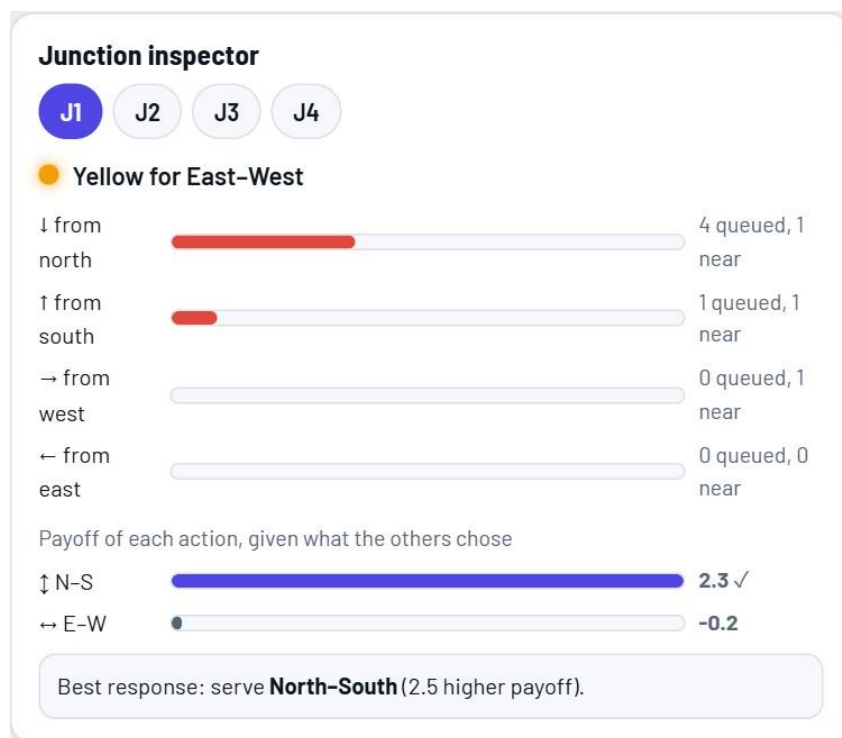


Figure 7: Junction inspector: queues per approach, payoff of each action and the reason for the decision

Dynamic features



Figure 8: A blocked approach at J2 (striped barrier with countdown); the junction no longer wastes green on it



Figure 9: An ambulance (flashing lights, red halo) approaching J1 from the west under preemption

Disrupt the network live · the same event hits all three controllers

Block an approach at the selected junction

from the west

For 180 s

Block approach (B) J2 west approach blocked

Ambulance enters the network

Row 2, from the west

Send ambulance (A) ☒ Emergency preemption (Selfish and Nash)

Latest ambulance trip, dispatch to exit. One run is noisy: send several, or use the Benchmark tab for an average over seeds.

Fixed-time: 64 s (mean 62 s, 4 trips)

Selfish: 35 s (mean 37 s, 5 trips)

Nash game: 39 s (mean 38 s, 5 trips)

Preemption is on for Selfish and Nash. Fixed-time cannot sense the ambulance.

Figure 10: Ambulance panel after five runs: mean trip times of the three controllers

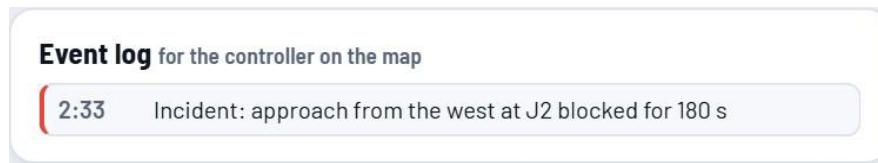


Figure 11: Event log of the incident

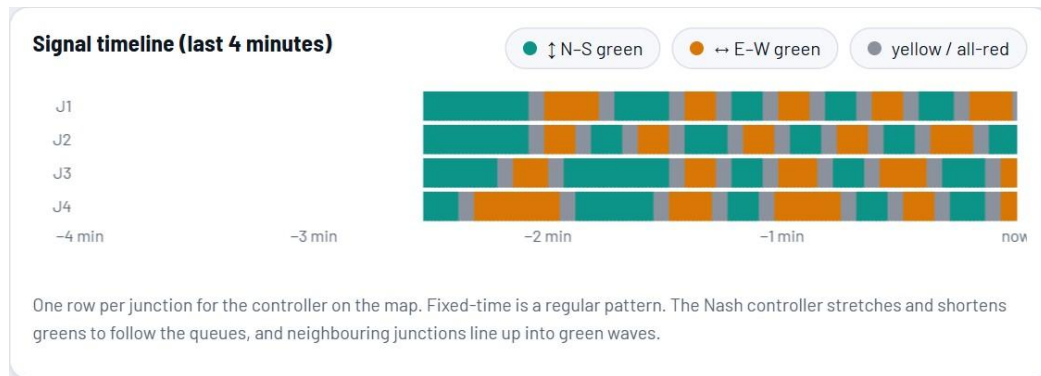


Figure 12: Signal timeline for the Nash controller: greens stretch and shrink with demand

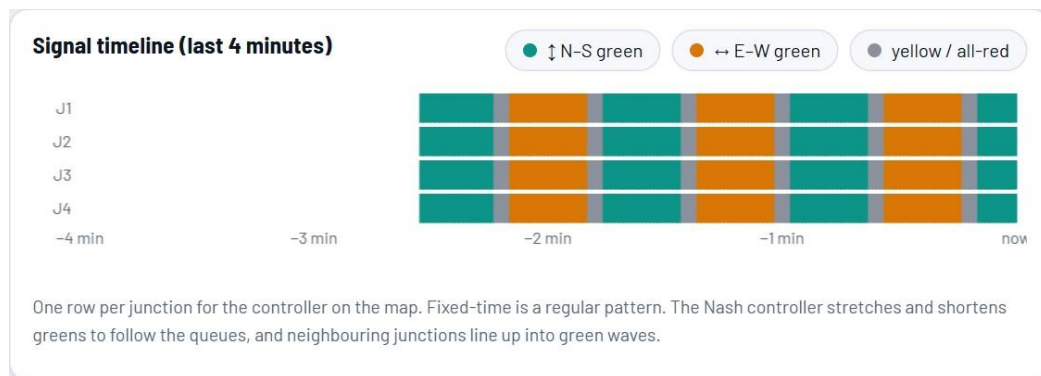


Figure 13: Signal timeline for the fixed-time controller: a regular pattern



Figure 14: The 3×3 grid with nine players

Game model

The signal game

Players. Each signalised junction is one rational agent. A 2×2 grid has 4 players, a 3×3 grid has 9.

Strategies. Player i picks which road gets the green next: $\updownarrow$ **North-South** or $\leftrightarrow$ **East-West**.

Payoff. The player gains for every vehicle it can clear, and for waiting time it stops from piling up. It loses for switching (lost time), for pushing cars into a neighbour whose queue is already full, and it gains when a neighbour is about to send it a platoon on the same road.

$$u_i(a_i, a_{-i}) = \sum_{k \in a_i} (q_k + 0.4 \cdot m_k + \omega \cdot w_k / 30) - \sigma \cdot 1[a_i \neq \text{current}] \\ - \lambda \sum_{j \in \text{down}(i, a_i)} q_{j \leftarrow i} \cdot (1 - 0.7 \cdot 1[a_j = a_i]) + \kappa \sum_{j \in \text{up}(i, a_i)} 0.6 \cdot q_{j \rightarrow i} \cdot 1[a_j = a_i]$$

q = stopped vehicles near the stop line, m = vehicles still approaching, w = total time the queue has waited, σ = switching cost, λ = penalty for spilling into a neighbour, κ = bonus for green-wave coordination.

Equilibrium. The signals run *best-response dynamics* once per second: each junction in turn picks its best action given the others' current choice, until nobody wants to change. That fixed point is a **pure-strategy Nash equilibrium**: no junction can do better by changing its own phase alone. Safety rules still apply: minimum green 8 s, maximum green 45 s, 2.5 s yellow, 1 s all-red.

Selfish = the same game with $\lambda = \kappa = 0$, so each junction ignores its neighbours. Fixed-time = a 20 s / 20 s timer.

Figure 15: Game model tab: players, strategies, payoff and equilibrium

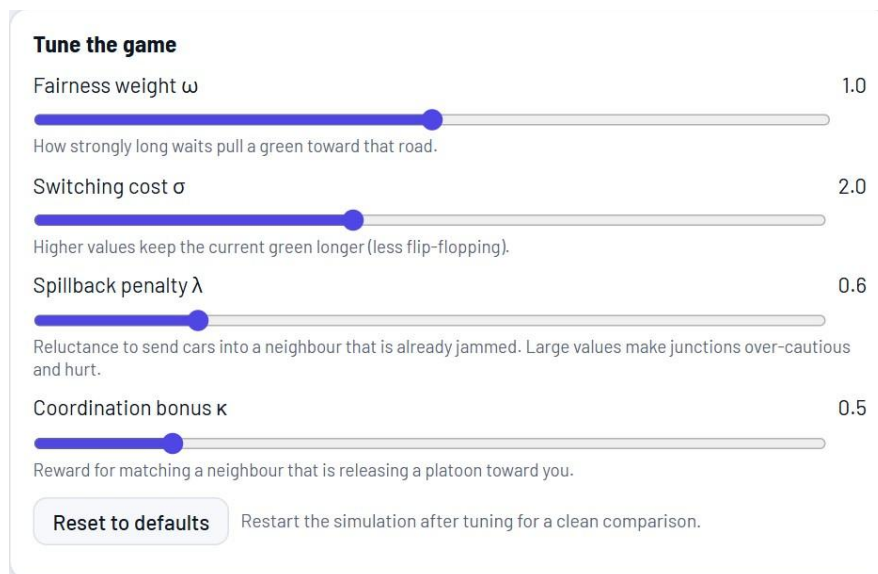


Figure 16: Live tuning of ω , σ , λ and κ

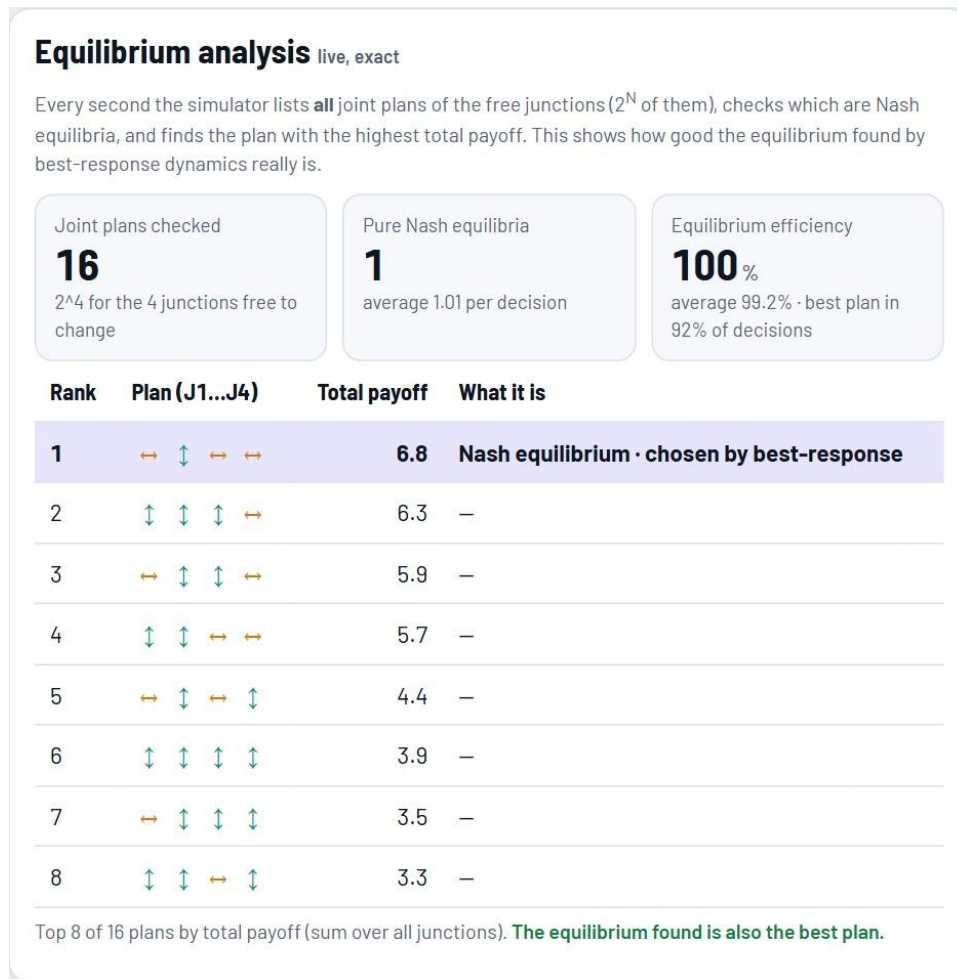


Figure 17: Equilibrium analysis: all 16 joint plans of a 2×2 grid ranked by total payoff

Benchmark and evidence

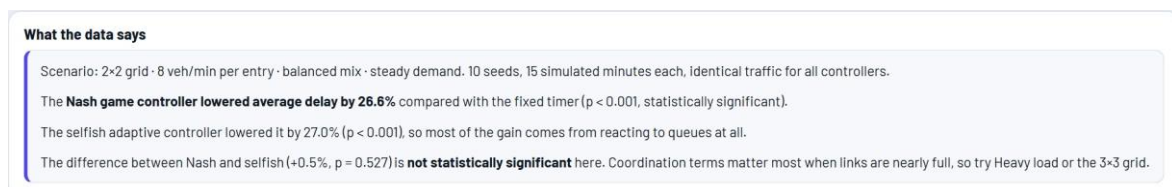


Figure 18: Benchmark summary written in plain language from the data

Results (mean ± standard deviation over seeds)					
Metric	Fixed-time	Selfish	Nash game	Nash vs fixed	Nash vs selfish
Average delay (s per vehicle)	34.6 ± 3.6	25.2 ± 2.2	25.4 ± 2.4	-26.6% $p < 0.001$	+0.5% $p = 0.527$
95th percentile delay (s)	83.0 ± 9.8	55.1 ± 4.9	55.9 ± 5.2	-32.7% $p < 0.001$	+1.4% $p = 0.538$
Average vehicles stopped	25.7 ± 3.0	17.6 ± 2.0	17.9 ± 2.1	-30.4% $p < 0.001$	+1.5% $p = 0.099$
Stops per trip	4.93 ± 0.58	4.42 ± 0.33	4.36 ± 0.39	-11.7% $p < 0.001$	-1.4% $p = 0.101$
Throughput (trips per minute)	61.3 ± 1.8	62.1 ± 1.7	62.2 ± 1.8	+1.4% $p < 0.001$	+0.1% $p = 0.601$
Idling time (vehicle-hours)	6.43 ± 0.75	4.41 ± 0.49	4.47 ± 0.52	-30.4% $p < 0.001$	+1.5% $p = 0.099$
Est. CO ₂ from idling (kg, illustrative)	11.9 ± 1.4	8.1 ± 0.9	8.3 ± 1.0	-30.4% $p < 0.001$	+1.5% $p = 0.099$
Signal switches per junction per hour	148 ± 0	265 ± 6	264 ± 6	+78.2% $p < 0.001$	-0.4% $p = 0.280$

Delay counts every vehicle: finished trips, cars still on the map, and cars still waiting to enter. A controller cannot look good by leaving cars stuck outside. Bold marks the best value in each row. Paired t-test across seeds: $p < 0.05$ is treated as significant.

Figure 19: Benchmark results table with paired t-tests (10 seeds, 15 minutes, 2×2 balanced)

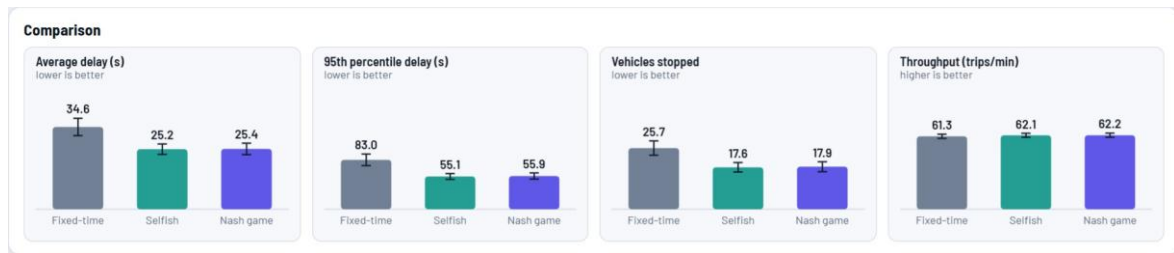


Figure 20: Comparison bars with standard deviation

Results (mean $\pm$ standard deviation over seeds)

Metric	Fixed-time	Selfish	Nash game	Nash vs fixed	Nash vs selfish
Average delay (s per vehicle)	47.4 $\pm$ 5.9	35.9 $\pm$ 6.4	35.6 $\pm$ 6.3	-24.8% p<0.001	-0.7% p=0.690
95th percentile delay (s)	150.8 $\pm$ 35.9	109.0 $\pm$ 25.0	106.5 $\pm$ 23.8	-29.4% p<0.001	-2.3% p=0.530
Ambulance trip time (s)	65.2 $\pm$ 12.3	42.4 $\pm$ 7.8	42.8 $\pm$ 6.5	-34.3% p<0.001	+1.0% p=0.577
Average vehicles stopped	30.9 $\pm$ 2.2	23.6 $\pm$ 3.9	23.8 $\pm$ 4.0	-23.0% p<0.001	+0.6% p=0.660
Stops per trip	5.31 $\pm$ 0.41	5.15 $\pm$ 0.57	5.08 $\pm$ 0.63	-4.4% p=0.110	-1.5% p=0.234
Throughput (trips per minute)	59.9 $\pm$ 1.5	61.2 $\pm$ 1.4	61.3 $\pm$ 1.4	+2.2% p<0.001	+0.1% p=0.749
Idling time (vehicle-hours)	7.72 $\pm$ 0.55	5.91 $\pm$ 0.98	5.95 $\pm$ 1.00	-23.0% p<0.001	+0.6% p=0.660
Est. CO ₂ from idling (kg, illustrative)	14.3 $\pm$ 1.0	10.9 $\pm$ 1.8	11.0 $\pm$ 1.8	-23.0% p<0.001	+0.6% p=0.660
Signal switches per junction per hour	148 $\pm$ 0	258 $\pm$ 4	257 $\pm$ 6	+73.3% p<0.001	-0.5% p=0.322

Delay counts every vehicle: finished trips, cars still on the map, and cars still waiting to enter. A controller cannot look good by leaving cars stuck outside. Bold marks the best value in each row. Paired t-test across seeds; p < 0.05 is treated as significant.

Figure 21: Benchmark with blocked approaches and ambulance runs

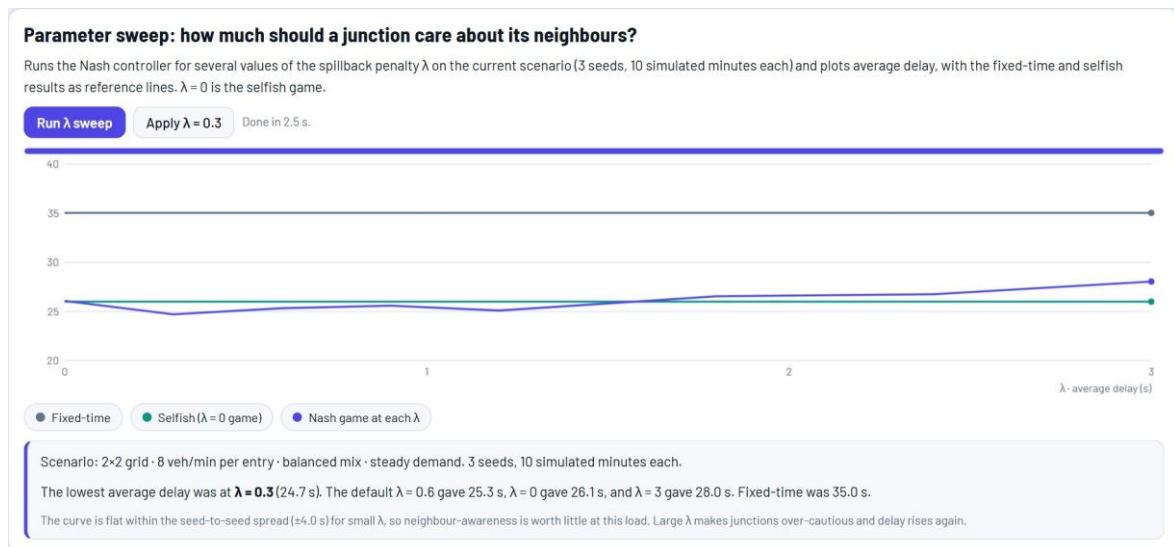


Figure 22: Parameter sweep of the spillback penalty λ

Run history Clear

When	Scenario	Seeds $\times$ min	λ	Fixed (s)	Selfish (s)	Nash (s)	Nash vs fixed	Nash vs selfish
11:57 PM	2x2 grid · 8 veh/min per entry · balanced mix · steady demand · blocked approaches and ambulance runs	10 $\times$ 15	0.6	47.4	35.9	35.6	-24.8% p<0.001	-0.7% p=0.690
11:57 PM	2x2 grid · 8 veh/min per entry · balanced mix · steady demand	10 $\times$ 15	0.6	34.6	25.2	25.4	-26.6% p<0.001	+0.5% p=0.527

Newest first, up to 20 runs. Stored locally in your browser.

Figure 23: Run history stored in the browser

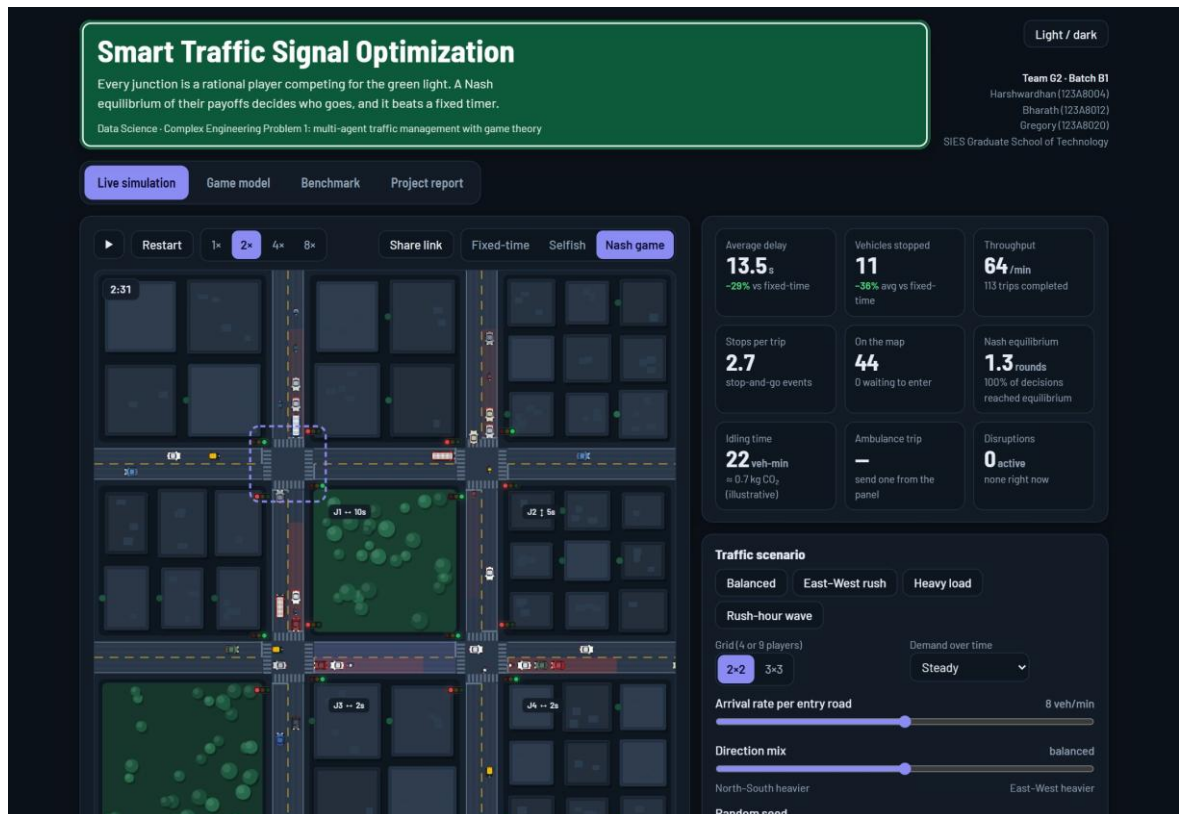


Figure 24: Dark theme

CHAPTER 8

Testing and Results

8.1 Test Strategy

Table 15: Automated tests

Test	Result	What is checked
Invariants (tests/inv.js)	0 overlaps	Vehicles never overlap (minimum gap 5 px kept) on all three controllers, 3×3 grid, 10 simulated minutes.
Invariants (tests/inv2.js)	0 violations	No vehicle crosses a stop line while its signal is red; crossings are counted: fixed 484, selfish 176, Nash 192.
Regression benchmark (tests/test.js)	unchanged	Delays of the three controllers are identical to the first version when no disruption is active.
Feature checks (tests/features.js)	17 / 17 pass	No crossing of a blocked stop line; incidents expire; Nash beats fixed under an incident; every ambulance trip completes; preemption beats fixed-time and beats the same controller without it; at least one Nash equilibrium found; efficiency within [0, 1]; analysis does not alter the simulation.
Experiments (tests/experiments.js)	about 500 runs	Every table in this chapter; also the source of the report numbers.

8.2 Controllers on Six Traffic Scenarios

Table 16: Average delay per vehicle, mean over 10 seeds (15 simulated minutes)

Scenario	Fixed (s)	Selfish (s)	Nash (s)	Nash vs fixed	Nash vs selfish
2x2 balanced (8 veh/min)	34.6	25.2	25.4	−26.6% (p < 0.001)	+0.5% (p 0.527)
2x2 east-west rush (+45%)	78.5	29.7	28.7	−63.4% (p < 0.001)	−3.2% (p 0.179)
2x2 heavy load (11 veh/min)	100.6	87.3	87.2	−13.3% (p < 0.001)	−0.1% (p 0.902)
2x2 rush-hour wave	112.2	99.9	99.8	−11.0% (p < 0.001)	−0.1% (p 0.876)
3x3 balanced (8 veh/min)	46.9	38.0	37.5	−20.0% (p < 0.001)	−1.4% (p 0.287)
3x3 heavy load (11 veh/min)	117.9	109.5	108.0	−8.4% (p < 0.001)	−1.3% (p 0.142)

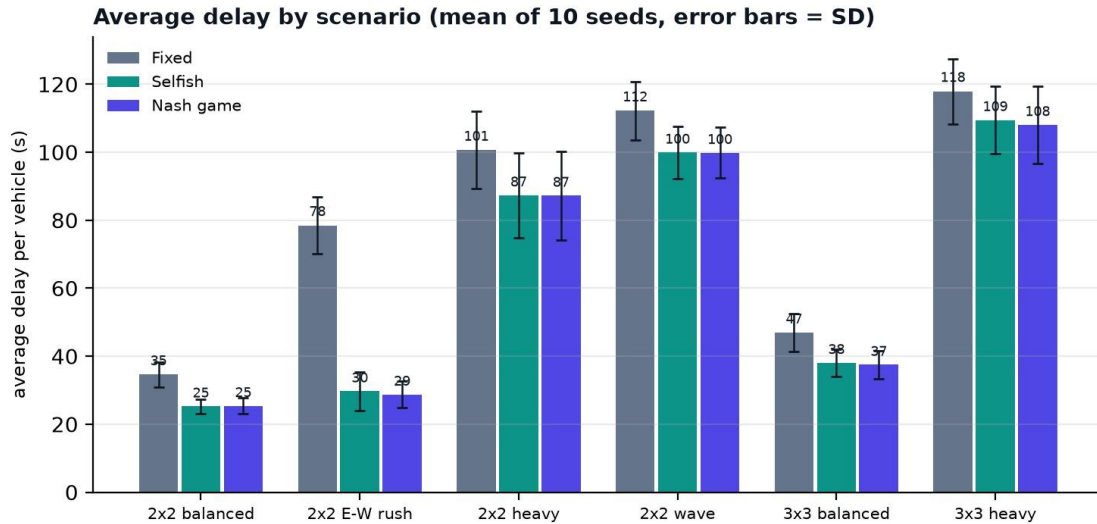


Figure 25: Average delay by scenario and controller

Both adaptive controllers beat the fixed timer in every scenario. The Nash controller lowered average delay by 8.4% to 63.4%, and every paired t-test has $p < 0.001$ (the largest p-value is $2.1e-5$). The gain is largest when one direction is much busier than the other (East–West +45%: 63.4%), where a fixed 20 s / 20 s split is badly mismatched. It is smallest under heavy load and in the rush-hour wave, where the network is close to saturation and there is little slack for any controller to exploit.

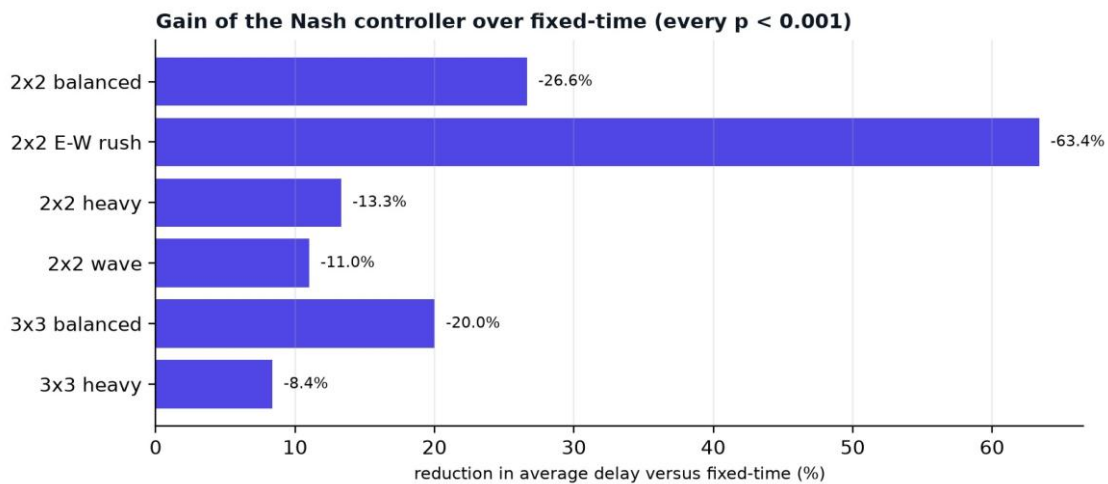


Figure 26: Reduction in average delay of the Nash controller relative to fixed-time

The Nash game versus the selfish controller

The coupled Nash game was not significantly better than the selfish adaptive controller in any scenario. The difference in average delay ranged from -3.2% to $+0.5\%$, with p-values between 0.14 and 0.90. We therefore draw the following conclusions and no stronger ones: (1) most of the benefit over fixed-time comes from reacting to queues at all; (2) the spillback and coordination terms did not measurably help at these loads on grids of this size, where links are short but rarely blocked; (3) the value of the game formulation here lies in the decision rule

being explainable, equilibria being checkable (Section 8.7), and the extension to preemption and incidents fitting naturally into the payoff.

8.3 Detailed Results for the Balanced 2×2 Scenario

Table 17: All metrics for 2×2 balanced, mean ± SD over 10 seeds

Metric	Fixed-time	Selfish	Nash game
Average delay (s)	34.6 ± 3.6	25.2 ± 2.2	25.4 ± 2.4
95th percentile delay (s)	83.0 ± 9.8	55.1 ± 4.9	55.9 ± 5.2
Vehicles stopped (average)	25.7 ± 3.0	17.6 ± 2.0	17.9 ± 2.1
Stops per trip	4.93 ± 0.58	4.42 ± 0.33	4.36 ± 0.39
Throughput (trips/min)	61.3 ± 1.8	62.1 ± 1.7	62.2 ± 1.8
Idling time (vehicle-hours)	6.43 ± 0.75	4.41 ± 0.49	4.47 ± 0.52
Est. CO ₂ from idling (kg, illustrative)	11.9 ± 1.4	8.1 ± 0.9	8.3 ± 1.0

Relative to fixed-time the Nash controller reduced the 95th percentile delay by 32.7%, so the improvement is not bought by sacrificing the slowest vehicles. Idling time fell by 30.4%, which corresponds to the illustrative CO₂ figures in the table. Adaptive controllers switch signals much more often than the fixed timer (about 265 switches per junction per hour versus 148) which is a cost in wear and driver expectation that the delay figures do not show.

8.4 Disruptions

Table 18: Average delay with disruptions injected identically for all controllers (2×2 balanced, 10 seeds)

Disruption	Fixed (s)	Selfish (s)	Nash (s)	Nash vs fixed
None (2×2 balanced)	34.6	25.2	25.4	−26.6%
Blocked approaches	46.8	33.7	34.0	−27.3%
Ambulance runs	34.7	26.3	26.4	−24.1%
Both	47.4	35.9	35.6	−24.8%

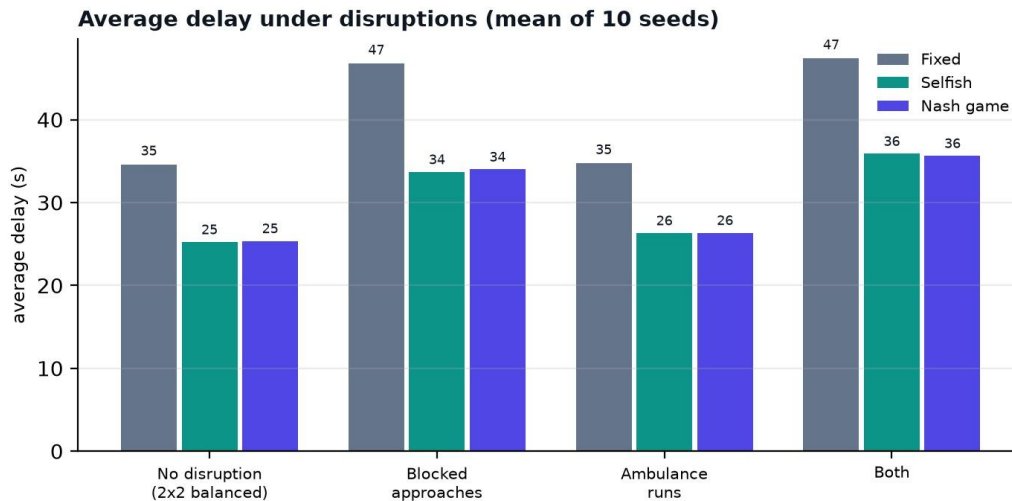


Figure 27: Average delay under disruptions

Blocked approaches raised delay for everyone, since vehicles on a blocked approach simply cannot move. The increase in average delay was +12.2 s for fixed-time, +8.5 s for selfish and +8.6 s for Nash: the adaptive controllers absorbed the disruption with about 29% less extra delay, because they stopped giving green to a stop line that could not use it. The relative advantage over fixed-time (−27.3%) stayed close to the undisturbed value (−26.6%): the disruption hurt everybody, and the adaptive controllers started from, and kept, a lower level.

8.5 Emergency Vehicle Preemption

Table 19: Ambulance trip time, three runs per seed at 180 s, 480 s and 720 s (10 seeds)

Configuration	Ambulance trip (s)	Comment
Fixed-time	50.1 ± 11.0	No sensing
Nash, preemption off	43.1 ± 9.3	Treats the ambulance like any vehicle
Nash, preemption on	31.8 ± 4.9	−26.2% vs off (paired $p < 0.001$)

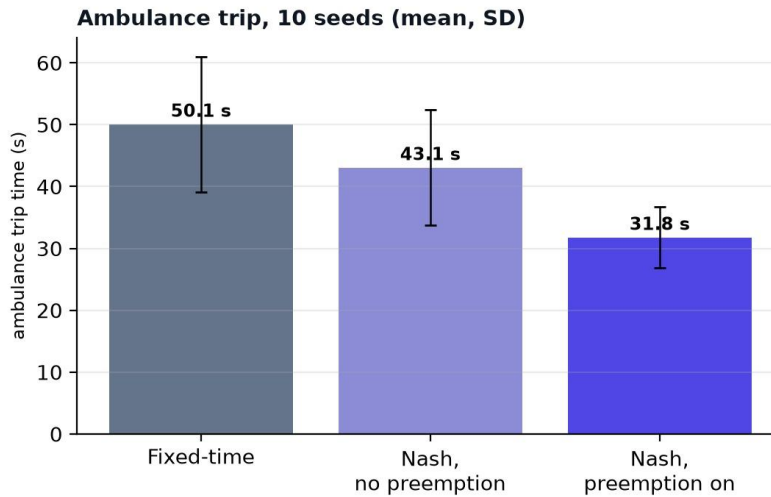


Figure 28: Ambulance trip time with and without preemption

Preemption cut the mean ambulance trip by 26% compared with the same controller without it, and by 37% compared with fixed-time. Without preemption the Nash controller is only slightly better than fixed-time for the ambulance (43.1 s versus 50.1 s), because it does not know the vehicle is special. The trip time includes any wait to enter the network. The single-run view on the live tab is noisy: one ambulance can be lucky with fixed-time signals, which is why the application reports a running mean and the Benchmark tab averages over seeds.

8.6 Sensitivity to the Spillback Penalty λ

Table 20: Average delay of the Nash controller against λ (mean $\pm$ SD over 10 seeds; fixed-time: 34.6 s and 117.9 s)

λ	2×2 balanced: delay (s)	3×3 heavy: delay (s)
0.0	25.3 $\pm$ 1.9	107.9 $\pm$ 11.2
0.3	25.0 $\pm$ 2.1	107.6 $\pm$ 10.8
0.6	25.4 $\pm$ 2.4	108.0 $\pm$ 11.4
0.9	25.2 $\pm$ 2.1	107.4 $\pm$ 10.3
1.2	25.5 $\pm$ 2.5	107.2 $\pm$ 9.0
1.8	26.5 $\pm$ 2.1	107.4 $\pm$ 10.5
2.4	27.4 $\pm$ 2.7	110.7 $\pm$ 9.9
3.0	29.6 $\pm$ 3.2	113.7 $\pm$ 11.0

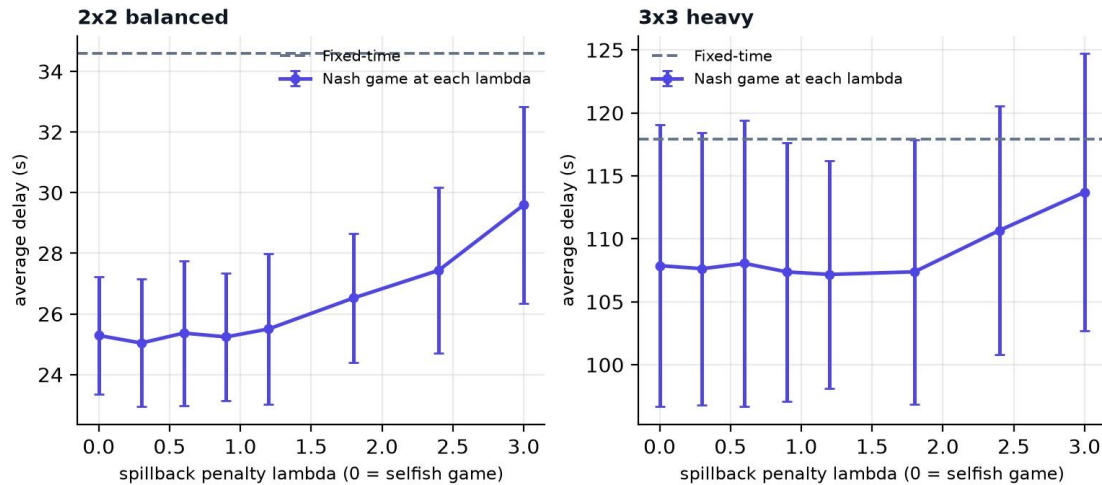


Figure 29: Delay against the spillback penalty λ

For both scenarios the curve is flat for λ up to about 1.2 (the lowest means are at $\lambda = 0.3$ and $\lambda = 1.2$, and the differences are well inside the seed-to-seed spread). For large λ delay rises: at $\lambda = 3$ delay on the 2×2 grid is 29.6 s against 25.3 s at $\lambda = 0$ (+17.0%), because junctions become over-cautious and hold back vehicles to protect neighbours. This is the trade-off between private and network benefit: neighbour-awareness is harmless in moderation and harmful in excess.

8.7 Quality of the Equilibrium

Table 21: Exact enumeration of all joint plans every second (10 seeds, 15 minutes)

Scenario	Converged	Pure NE per decision	Efficiency / best plan
2x2 balanced	100.0%	1.03	98.2% / 84%
3x3 balanced	100.0%	1.08	98.3% / 64%
3x3 heavy	100.0%	1.20	97.1% / 57%

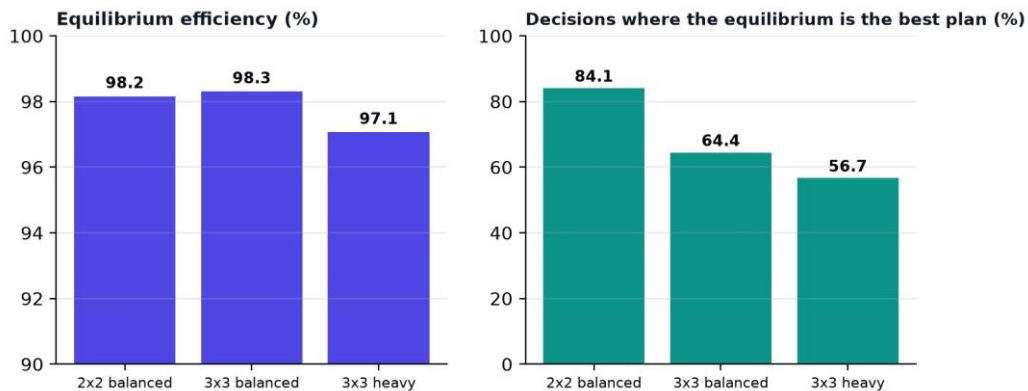


Figure 30: Equilibrium efficiency and share of decisions where the equilibrium is the best plan

Best-response dynamics reached an equilibrium in every decision. There was almost always exactly one pure equilibrium (1.03 to 1.20 on average), so the equilibrium selection problem is mild for this payoff design. The equilibrium found was the best joint plan in 57% to 84% of decisions, and on average reached 97.1% to 98.3% of the range between the worst and the best plan. The share of best-plan decisions falls as the grid grows and loads rise, as expected: with more players and more coupling the equilibrium can differ from the optimum.

8.8 Threats to Validity and Limitations

- **Synthetic demand.** Arrivals are Poisson and the network is a regular grid; real traffic has platoons, pedestrians and irregular layouts. The numbers describe controllers inside our simulator.
- **Simplified physics.** One lane per direction, a simple car-following rule and an abstract scale; no calibration against detector data and no validation against SUMO.
- **Two phases.** No left-turn phases, no pedestrian phases, no offsets between junctions.
- **Small grids and a single seed set.** Results are for 2×2 and 3×3 grids and ten seeds; effects too small to detect with ten seeds may exist (for example Nash versus selfish).
- **Equal treatment of vehicles.** The payoff counts vehicles, not people; a bus counts as one vehicle.
- **Disruption model.** An incident is a complete blockage of one approach for a fixed time, known to the controller through its detector.
- **CO₂ figure.** An illustration from stated assumptions, not a measurement.

CHAPTER 9

Social Impact, Ethics and Sustainability

9.1 Expected Impact

- **Less time lost.** In simulation the adaptive controllers cut delay by 8% to 63%, depending on load.
- **Faster emergency response.** Ambulance trips are about 37% shorter than with fixed-time.
- **Resilience.** Controllers that notice a blocked road stop wasting green on it.
- **Transparency.** The payoff matrix and equilibrium analysis let an operator see why a signal chose a phase.

9.2 Ethical Considerations

- **Safety first.** No controller can break yellow and all-red clearance; preemption shortens only the minimum green.
- **Fairness between roads.** The fairness term ω prevents a minor road from waiting indefinitely, and a maximum green of 45 s caps any single phase. The 95th-percentile delay is reported so that improvements are not bought at the expense of the slowest vehicles.
- **Equal treatment of road users.** The payoff counts vehicles; a deployment should consider weighting buses, cyclists and pedestrians.
- **Privacy.** The system uses only anonymous vehicle counts; it stores and transmits no personal data.
- **Honest reporting.** The Nash-versus-selfish result is reported as it is, and the synthetic nature of demand is stated throughout.

9.3 Sustainability

Reducing idling lowers fuel use and local emissions. In the balanced 2×2 scenario the Nash controller cut idling by 30%. The software itself is lightweight: the full set of about 500 simulated runs behind this report completes in under three minutes on a laptop, and the application runs in a browser without a server.

CHAPTER 10

Mapping to Course Outcomes, Programme Outcomes, Knowledge Profiles and WP

This chapter relates the project to the outcomes of the Data Science course and to the programme outcomes (PO), knowledge profiles (WK) and complex problem-solving attributes (WP) used for Complex Engineering Problems. Course-outcome numbers follow the order of topics in the course; adjust them to the official list if it differs.

10.1 Course Outcome Mapping

Table 22: Course outcome mapping

CO	Outcome (summary)	Where it is demonstrated
CO1	Generate, collect and prepare data for analysis.	Chapter 6: seeded synthetic demand, common random numbers, metric definitions.
CO2	Explore and summarise data with descriptive statistics and visualisation.	Chapter 8 and Chapter 7: means, standard deviations, percentiles, charts, timelines.
CO3	Build and apply models and algorithms to a problem.	Chapter 5: game model, best-response dynamics, equilibrium enumeration.
CO4	Evaluate models and draw statistically sound inferences.	Chapter 8: paired t-tests, sweeps, honest handling of non-significant results.
CO5	Build, present and communicate a data-driven application for a real-world problem.	Chapters 4, 7: complete application; report; presentation script.

10.2 Programme Outcome Mapping

Table 23: Programme outcome mapping

PO	Programme outcome	Evidence	Level
PO1	Engineering knowledge	Game theory, traffic flow, probability and statistics, algorithms.	High
PO2	Problem analysis	Decomposed the problem into players, strategies, payoffs, safety rules and metrics (Chapters 3, 5).	High
PO3	Design and development of solutions	Architecture, controllers, simulator, interface (Chapters 4 to 7).	High
PO4	Investigation of complex problems	Controlled experiments, sweeps, equilibrium analysis, threats to validity (Chapter 8).	High

PO	Programme outcome	Evidence	Level
PO5	Modern tool usage	JavaScript, Node.js, Python, Playwright, Matplotlib, Git and GitHub.	High
PO6	The engineer and society	Congestion, emergency response, SDG 11 (Chapters 1.5, 9).	High
PO7	Environment and sustainability	Idling reduction and emissions estimate (Chapter 9.3).	Medium
PO8	Ethics	Safety-first design, fairness, honest reporting (Chapter 9.2).	Medium
PO9	Individual and team work	Three-member team with a shared repository.	Medium
PO10	Communication	This report, presentation script, README, in-app report.	High
PO11	Project management and finance	Work breakdown, version control, automated tests, no paid services (Chapter 3.5).	Medium
PO12	Life-long learning	Self-learning of game theory, simulation and statistics beyond the syllabus.	Medium

10.3 Knowledge Profile (WK) Mapping

Table 24: Knowledge profile mapping

WK	Knowledge profile	Evidence
WK2	Mathematics, statistics and computer science	Probability (Poisson arrivals), game theory, hypothesis testing, algorithms.
WK3	Engineering fundamentals	Vehicle kinematics (safe-stopping speed), queueing behaviour at signals.
WK4	Specialist engineering knowledge	Traffic signal control, multi-agent systems, simulation.
WK5	Engineering design	System architecture, decision cycle, interface design.
WK6	Engineering practice	Testing, version control, reproducible experiments.
WK7	Engineering in society	Urban mobility, emergency services, SDG 11.
WK8	Research literature	Webster, max-pressure, potential games, multi-agent traffic control (References).
WK9	Ethics, inclusive behaviour and conduct	Safety, fairness, privacy, honest reporting of limitations.

10.4 Complex Problem-Solving Attributes (WP) Mapping

The complete justification of WP1 to WP7 is in Chapter 3.4. In summary, WP1, WP2, WP3, WP4, WP6 and WP7 are strongly addressed and WP5 is partly addressed: signal clearance practice applies, but there is no code of practice for the simulation model itself.

CHAPTER 11

Conclusion and Future Scope

11.1 Conclusion

This project set out to show that traffic signals can be treated as rational players in a game, that the equilibrium of that game is cheap enough to compute every second, and that the resulting control beats a fixed timer. In our simulator it does: across six scenarios the Nash controller reduced average delay by 8% to 63% with every paired t-test below 0.001, and it kept the 95th-percentile delay down as well. Version 2 made the system dynamic: users can block a road or send an ambulance while it runs, and the controllers respond; with preemption the ambulance trip fell from 43.1 s to 31.8 s. An exact enumeration of all joint plans showed that best-response dynamics reached an equilibrium every time, that the equilibrium was nearly always unique, and that it captured 97% to 98% of the best possible joint payoff.

The work also made limits clear. The coupled Nash game was not significantly better than the selfish adaptive controller in any scenario, so we attribute the gain over fixed-time to adaptivity, not to coordination. Demand is synthetic and the model is not validated against real data, so the figures describe the controllers in our simulator and not a real road network. The value of the project is therefore the complete, explainable, testable and reproducible system, together with an honest statement of what it does and does not show.

11.2 Future Scope

- Calibrate demand and parameters with real detector or video counts from a Navi Mumbai corridor, and validate against SUMO.
- Larger networks with heterogeneous link lengths, multiple lanes, left-turn and pedestrian phases, and offsets between junctions.
- Learn the payoff weights with multi-agent reinforcement learning, and compare with max-pressure control [3].
- Bayesian games with uncertain demand; priorities for buses and non-motorised road users.
- Test whether coordination terms pay off on larger grids, with link capacities closer to the limit, using a much larger number of seeds.
- Connect to live detectors and a real signal controller through a safe supervisory interface.

References

- [1] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," General Assembly resolution A/RES/70/1, 2015.
- [2] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, HMSO, London, 1958.
- [3] P. Varaiya, "Max pressure control of a network of signalized intersections," Transportation Research Part C, vol. 36, pp. 177-195, 2013.
- [4] J. Nash, "Equilibrium points in n-person games," Proceedings of the National Academy of Sciences, vol. 36, no. 1, pp. 48-49, 1950.
- [5] D. Monderer and L. S. Shapley, "Potential games," Games and Economic Behavior, vol. 14, no. 1, pp. 124-143, 1996.
- [6] A. L. C. Bazzan, "Opportunities for multiagent systems and multiagent reinforcement learning in traffic control," Autonomous Agents and Multi-Agent Systems, vol. 18, pp. 342-375, 2009.
- [7] S. Krauss, P. Wagner and C. Gawron, "Metastable states in a microscopic model of traffic flow," Physical Review E, vol. 55, no. 5, pp. 5597-5602, 1997.
- [8] A. M. Law, Simulation Modeling and Analysis, 5th ed. New York: McGraw-Hill, 2015.
- [9] D. C. Montgomery and G. C. Runger, Applied Statistics and Probability for Engineers, 7th ed. Hoboken, NJ: Wiley, 2018.
- [10] T. Roughgarden and É. Tardos, "How bad is selfish routing?" Journal of the ACM, vol. 49, no. 2, pp. 236-259, 2002.
- [11] Y. Shoham and K. Leyton-Brown, Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations. Cambridge University Press, 2008.
- [12] Indian Roads Congress, "Guidelines on Design and Installation of Road Traffic Signals," IRC:93.
- [13] Project source code, GitHub repository. <https://github.com/bharath050305/traffic-signal-game-theory>

Appendix A: Running the Project

Source code: <https://github.com/bharath050305/traffic-signal-game-theory>

The application needs no installation. Open index.html in a browser. To rebuild after editing the sources, and to run the tests, open a terminal in the project folder:

```
python build.py
node tests/features.js
node tests/experiments.js
```

Table 25: Commands

Command	Effect
python build.py	Bundle src/ into index.html.
node tests/test.js '{"n":2,"rate":8,"bias":0}'	Headless benchmark of the three controllers (5 seeds).
node tests/inv.js ; node tests/inv2.js	Overlap and red-light invariants.
node tests/features.js	17 checks of incidents, preemption and equilibrium analysis.
node tests/experiments.js	All experiments in the report; writes docs/results.json (about 3 minutes).
python docs/capture_screenshots.py	Recaptures the report screenshots (needs a local web server and Playwright).

Appendix B: Parameters

Table 26: Parameters

Parameter	Value	Meaning
Time step	0.1 s	Simulation step.
Decision interval	1 s	The controllers decide once per simulated second.
Yellow / all-red	2.5 s / 1.0 s	Clearance between phases.
Minimum / maximum green	8 s / 45 s	Safety limits for adaptive control.
Preemption minimum green	3 s	Only for a junction holding an ambulance back.
Fixed-time green	20 s each	Baseline plan.
Detector ranges	150 px / 260 px	Queue detection and approach detection (about 45 m and 78 m).
$\omega, \sigma, \lambda, \kappa, E$	1.0, 2.0, 0.6, 0.5, 60	Payoff weights.
Free speed, acceleration, braking	45, 16, 26 px/s (px/s ²)	Vehicle dynamics.
Turning probability	0.25	Share of vehicles that turn at a junction.
Best-response cap	8 rounds	Cut-off for cycling dynamics.
Idle fuel, CO ₂ factor	0.8 L/h, 2.31 kg/L	Assumptions for the illustrative emissions estimate.